

```
$ onboard langchain-ai/langgraph --profile dev
```

# LangGraph : le framework d'agents à état de LangChain, un monorepo Python de 9 paquets piloté au Makefile, où votre première PR passe par `make test` dans la lib touchée

Build resilient agents.

**41 386**

ÉTOILES GITHUB

**10**

CONTRIBUTEURS ACTIFS  
sur 12 semaines

**245**

PR OUVERTES

**9 sept. 2026**

DERNIER COMMIT

## ## Que fait ce projet et pour qui ?

> Un framework bas niveau, Python, MIT, pour des agents à état qui survivent aux pannes : vous y contribuez si les mots graphe, checkpoint et LLM vous parlent

LangGraph se présente en trois mots, « Build resilient agents », et en une phrase dans le README : un framework d'orchestration bas niveau pour construire, gérer et déployer des agents à état, de longue durée<sup>1</sup>.

Le README revendique trois briques : l'exécution durable (l'agent reprend exactement où il s'est arrêté après une panne), l'humain dans la boucle (inspecter et modifier l'état en cours d'exécution) et la mémoire, courte pendant le raisonnement et longue entre les sessions.

Le public visé est le développeur Python qui construit des agents LLM. Le README cite Klarna, Replit et Elastic parmi les utilisateurs, et renvoie vers Deep Agents pour qui veut un niveau plus haut, vers LangGraph.js pour l'équivalent TypeScript.

Le projet est autonome mais s'intègre à LangChain (composants), LangSmith (traces, évaluations) et LangSmith Deployment (hébergement). Il est inspiré de Pregel et Apache Beam, avec une interface publique dans l'esprit de NetworkX.

| REPÈRE       | VALEUR                                                          |
|--------------|-----------------------------------------------------------------|
| Étoiles      | 41 384                                                          |
| Forks        | 6 990                                                           |
| Licence      | MIT                                                             |
| Créé le      | 2023-08-09                                                      |
| Propriétaire | organisation langchain-ai, branche par défaut <code>main</code> |
| Dernier push | 2026-09-09                                                      |

Parmi les 18 topics déclarés : agents, multiagent, llm, rag, python, pydantic, openai, gemini.

## Comment le code est-il organisé ?

> Un paquet = un dossier de `libs/` avec son `pyproject.toml` , son `Makefile` et son `tests/` ; `libs/langgraph` est le cœur, les huit autres gravitent autour

- └ `.github/` 17 workflows GitHub Actions, scripts CI
- └ `docs/` documentation
- └ `examples/` exemples d'usage, notebooks
- └ `libs/` code principal : 9 paquets, 8 en Python, sdk-js réduit à un README
- └ `.gitignore` exclusions git
- └ `.markdownlint.json` config lint Markdown
- └ `AGENTS.md` consignes pour les agents IA
- └ `CLAUDE.md` consignes pour Claude Code
- └ `LICENSE` licence MIT
- └ `Makefile` pilote du monorepo : install, lint, test via uv
- └ `README.md` présentation du projet

Onze entrées à la racine et aucun manifeste : tout le code vit dans `libs/` , un dossier par paquet publié, chacun avec son `pyproject.toml` , son `uv.lock` , son `Makefile` et son `tests/` ; le `Makefile` racine ne fait que déléguer à chaque lib.

| PAQUET                                                                         | RÔLE D'APRÈS AGENTS.MD                                                                            |
|--------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <code>libs/langgraph</code>                                                    | le cœur : agents à état, multi-acteurs, version 1.2.11                                            |
| <code>libs/checkpoint</code>                                                   | interfaces de base des checkpointers                                                              |
| <code>libs/checkpoint-postgres</code> ,<br><code>libs/checkpoint-sqlite</code> | implémentations Postgres et SQLite du checkpointer                                                |
| <code>libs/checkpoint-conformance</code>                                       | suite de conformité des checkpointers                                                             |
| <code>libs/prebuilt</code>                                                     | API haut niveau pour créer et lancer agents et outils                                             |
| <code>libs/cli</code>                                                          | la commande <code>langgraph</code> , exemples Python et JS, schéma de <code>langgraph.json</code> |
| <code>libs/sdk-py</code> , <code>libs/sdk-js</code>                            | SDK Python et JS/TS pour l'API LangGraph Server                                                   |

Le cœur dépend de trois paquets frères, `langgraph-checkpoint` , `langgraph-sdk` et `langgraph-prebuilt` , liés en editable via `[tool.uv.sources]` , plus `langchain-core` , `pydantic` et `xxhash` : 6 dépendances d'exécution sur 33 déclarées<sup>2</sup>.

Autour : 17 workflows dans `.github/` , des notebooks dans `examples/` , un `docs/` réduit à des redirections et un `llms.txt` (la doc vit sur `docs.langchain.com`). `AGENTS.md` et `CLAUDE.md` donnent aux agents IA la carte des dépendances entre libs et les commandes à lancer avant une PR<sup>3</sup>.

## ## Par où commencer à lire ?

> Commencez par `libs/langgraph/langgraph/` ( `graph/` , `pregel/` , `runtime.py` ), le `Makefile` racine ensuite : tout le reste en découle

Avant le code, dix minutes sur `AGENTS.md` : la carte des dépendances entre libs et les trois commandes attendues avant une PR. Le README pointe ensuite le quickstart et la référence d'API sur docs.langchain.com, et un cours gratuit sur LangChain Academy.

1. `libs/langgraph/langgraph/` : le paquet Python `langgraph` , le cœur. Les sous-dossiers `graph/` (construction du graphe), `pregel/` (la boucle d'exécution) et `runtime.py` sont ceux que la recherche fait remonter<sup>4</sup>
2. `Makefile` à la racine : `make install` crée un venv uv et installe chaque `libs/*` en editable ; `lint` , `format` , `lock` , `test` délèguent au `Makefile` de chaque lib<sup>5</sup>
3. `libs/cli/langgraph_cli/cli.py` : la commande `langgraph` , déclarée par `[project.scripts]` dans `libs/cli/pyproject.toml` , construite sur click<sup>6</sup>
4. `libs/cli/generate_schema.py` : génère `libs/cli/schemas/schema.json` , le schéma de `langgraph.json` ; la CI le relance et échoue si le schéma a changé sans commit
5. `.github/workflows/ci.yml` : l'orchestration de la CI, filtre par chemins modifiés puis lint et tests par lib

Les `TODO` laissés dans le cœur disent ce qui bouge : la signature des nœuds dans `libs/langgraph/langgraph/graph/_node.py` (« once we move to adding a context arg »), un gestionnaire de contexte pour le runtime dans `runtime.py` , un paramètre `exiting` explicite dans `pregel/_loop.py` <sup>7</sup>.

Donnée non relevée : le contenu des fichiers d'entrée eux-mêmes, les rôles ci-dessus sont déduits des manifestes et de la CI ; à vérifier dans <https://github.com/langchain-ai/langgraph/tree/main/libs/langgraph/langgraph>.

## ## Comment builder, tester et lancer ?

> `make install` à la racine, puis `make format && make lint && make test` dans la lib touchée : c'est exactement ce que `ci.yml` rejoue sur chaque PR

Pour utiliser la bibliothèque : `pip install -U langgraph`, Python 3.10 ou plus.

Pour y contribuer, le monorepo se pilote avec uv depuis le `Makefile` racine, puis lib par lib. `AGENTS.md` fixe la règle : formater, linter et tester dans le dossier de la lib modifiée avant toute PR.

```
make install                # uv venv, puis chaque libs/* en editable
cd libs/langgraph
make format && make lint && make test # ce que la CI rejoue
TEST=tests/test_algo.py make test  # un seul fichier, autres options pytest dans TEST
```

Les tests sont en pytest, avec `pytest-xdist` pour paralléliser, `syrrupy` pour les snapshots, et des options strictes ( `--strict-markers` `--strict-config` ). Le dossier est `libs/langgraph/tests`, `libs/cli/tests` pour la CLI.

Donnée non relevée : le nombre de fichiers de test ; à vérifier dans <https://github.com/langchain-ai/langgraph/tree/main/libs/langgraph/tests>.

Le lint est ruff (règles E, F, I, UP, TID251, ligne à 88, cible py310) et le typage ty ; la CLI ajoute codespell.

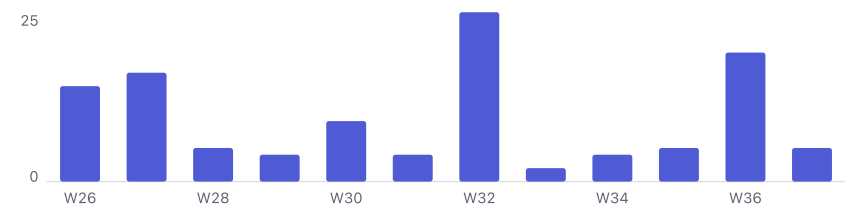
Pas de commande de lancement à proprement parler : c'est une bibliothèque, on l'importe. La commande `langgraph` de `libs/cli` sert à servir et déployer un projet, avec l'extra `inmem` pour un serveur local.

La CI, `.github/workflows/ci.yml`, part sur `push` vers `main`, sur chaque `pull_request` et à la demande. Elle filtre par chemins modifiés, puis lance le lint sur 8 libs, les tests sur 7 libs via `_test.yml` et sur `libs/langgraph` via `_test_langgraph.yml`, vérifie le schéma de la CLI, joue les tests d'intégration CLI et sdk-py, et un job `ci_success` agrège le tout. Actions épinglées par SHA, runs concurrents annulés<sup>8</sup>.

Donnée non relevée : pas de `CONTRIBUTING.md` dans le dépôt, le guide de contribution est externe ; à vérifier dans <https://docs.langchain.com/oss/python/contributing/overview>.

## Comment le projet vit-il ?

> Un dépôt vivant : dernier commit le 2026-09-09, une release de langgraph toutes les deux à quatre semaines cet été, quatre personnes pour la moitié des commits



Dernier commit sur main le 2026-09-09. Sur les 12 dernières semaines, le rythme est irrégulier : 25 commits la semaine 32, 2 la semaine suivante, 19 la semaine 36<sup>9</sup>.

Les commits sont répartis : eliornl en tête avec 11, puis ccurme et longquanzheng à 5 chacun ; il faut 4 personnes pour atteindre la moitié des commits de la période (bus factor 4).

Les releases sont par paquet, chacune avec son tag. Les 10 dernières relevées tiennent en sept semaines<sup>10</sup> :

| PAQUET                        | VERSIONS RÉCENTES     | DATES                              |
|-------------------------------|-----------------------|------------------------------------|
| langgraph                     | 1.2.9, 1.2.10, 1.2.11 | 2026-07-10, 2026-07-28, 2026-08-11 |
| langgraph-sdk                 | 0.4.3, 0.4.4          | 2026-08-19, 2026-08-27             |
| langgraph-checkpoint          | 4.2.0                 | 2026-08-07                         |
| langgraph-checkpoint-postgres | 3.1.1, 3.1.2          | 2026-07-30, 2026-08-07             |
| langgraph-checkpoint-sqlite   | 3.1.1                 | 2026-07-30                         |
| langgraph-cli                 | 0.4.31                | 2026-07-10                         |

Sur les 33 PR fusionnées en 30 jours, la majorité sont des montées de dépendances par dependabot ; les changements de fond sont ciblés : détection des sous-graphes depuis le bytecode (#8569), type de Store.put élargi (#8617), typage des projections de stream v3 (#8596), traces LangSmith depuis les streams (#8723)<sup>11</sup>.

Côté risques : licence MIT sans surprise, CI solide, dépendances maîtrisées (uv.lock versionné). Un point orange : pas de SECURITY.md , donc aucun canal déclaré pour signaler une faille.

## ## Quelle première contribution ?

> Pas d'étiquette « good first issue » : prenez un `TODO` de tests ou une issue bien cadrée comme #8130, avec un titre `fix(scope): ...` et un lien vers l'issue

Le volume est celui d'un gros projet : 528 issues ouvertes, 7 fermées sur les 30 derniers jours, 245 PR ouvertes. Les labels ne trient pas par difficulté : `external` 297, `bug` 81, `internal` 3, et aucune issue étiquetée « good first issue »<sup>12</sup>.

Le README renvoie au guide de contribution externe, <https://docs.langchain.com/oss/python/contributing/overview>, pour trouver de bonnes premières issues.

Trois pistes concrètes, tirées de ce qui est relevé :

- Une correction en une ligne : #8130, la coquille « GraphRecursionError » dans la docstring de `create_react_agent`, 17 commentaires, toujours ouverte<sup>13</sup>
- Des tests à compléter, signalés par les mainteneurs eux-mêmes : `# TODO: add more tests` dans `libs/langgraph/tests/test_algo.py`, `# TODO: test before and limit params` dans `libs/checkpoint/tests/test_memory.py`
- Un bug de sérialisation bien cadré : #8185, le checkpoint rejette `fractions.Fraction` et `complex` alors que `Decimal` passe, 11 commentaires<sup>14</sup>

Les sujets chauds sont ailleurs et demandent du contexte : reçus auditable de fin d'exécution (#7844, 64 commentaires), longs appels d'outils rejoués depuis le checkpoint (#7417, 51), un `ApprovalNode` pour l'humain dans la boucle (#8026, 45), ordre de persistance en mode `durability="sync"` (#8039, 36)<sup>15</sup>.

Les PR fusionnées suivent un titre `type(scope): sujet`, par exemple `fix(cli): clarify missing deploy config` ; les workflows `require_issue_link.yml` et `pr_lint.yml` existent dans `.github/workflows/`, non lus, leur nom suffit à deviner la règle.

Les PR ouvertes les plus récentes montrent où le cœur bouge : rejeu d'une branche abandonnée dans un `DeltaChannel` (#8548), `stream_mode="messages"` restreint à certaines clés d'état (#8868), et deux brouillons sur les sous-graphes parallèles et le pilotage d'un graphe en cours (#8819, #8838)<sup>16</sup>.

## \$ Vos 3 premières actions

1. Cloner, puis `make install` à la racine et `cd libs/langgraph && make test` : si la suite passe, votre environnement est celui de la CI
2. Lire `AGENTS.md` en entier, puis `libs/langgraph/langgraph/graph/`, `pregel/` et `runtime.py` avec les `TODO` sous les yeux : c'est là que la prochaine version se décide
3. Ouvrir une première PR sur #8130 ou un `TODO` de `tests/test_algo.py`, titre `fix(langgraph): ...`, issue liée, `make format && make lint && make test` verts dans la lib avant de pousser<sup>13</sup>



## RÉFÉRENCES PUBLIQUES

### ## Sources

1. <https://github.com/langchain-ai/langgraph/blob/main/README.md>
2. <https://github.com/langchain-ai/langgraph/blob/main/libs/langgraph/pyproject.toml>
3. <https://github.com/langchain-ai/langgraph/blob/main/AGENTS.md>
4. <https://github.com/langchain-ai/langgraph/tree/main/libs/langgraph/langgraph>
5. <https://github.com/langchain-ai/langgraph/blob/main/Makefile>
6. <https://github.com/langchain-ai/langgraph/blob/main/libs/cli/pyproject.toml>
7. <https://github.com/search?q=repo%3Alangchain-ai%2Flanggraph+TODO&type=code>
8. <https://github.com/langchain-ai/langgraph/blob/main/.github/workflows/ci.yml>
9. <https://github.com/langchain-ai/langgraph/commits/main>
10. <https://github.com/langchain-ai/langgraph/releases>
11. <https://github.com/langchain-ai/langgraph/pulls?q=is%3Apr+is%3Amerged>
12. <https://github.com/langchain-ai/langgraph/issues>
13. <https://github.com/langchain-ai/langgraph/issues/8130>
14. <https://github.com/langchain-ai/langgraph/issues/8185>
15. <https://github.com/langchain-ai/langgraph/issues/7844>
16. <https://github.com/langchain-ai/langgraph/pulls>